

Pichulman Miguel Angel

METODOLOGIA DE SISTEMAS

Trabajo Práctico 7 : Fundamentos y Técnicas de Diseño de Pruebas

Parte 1: Fundamentos y Mentalidad del Tester

1. Defina qué es el Testing de Software y explique la metáfora del "Costo Exponencial de un Defecto". ¿Por qué se considera una inversión y no un gasto?

El Testing de Software es el proceso de evaluar un sistema o sus componentes para verificar que cumplan con los requisitos especificados y funcionen correctamente. Su objetivo principal es medir la calidad del producto, reducir riesgos y confirmar que el software desarrollado satisface las necesidades del cliente.

El Costo Exponencial de un Defecto indica que cuanto más tarde se detecta un error, más costoso resulta corregirlo. Un defecto encontrado durante la etapa de requisitos puede solucionarse fácilmente, mientras que si se descubre cuando el sistema ya está en producción puede generar pérdidas económicas, problemas operativos y daños a la reputación de la empresa.

Por este motivo, el testing se considera una inversión y no un gasto, ya que permite detectar problemas tempranamente y evitar costos mucho mayores en etapas posteriores del proyecto.

2. Explique la "Mentalidad del Tester". Utilizando el ejemplo de un formulario de login, describa 3 preguntas (del tipo "¿Qué pasa si...?") que haría un tester y que un desarrollador podría pasar por alto.

La mentalidad del tester es investigativa y está basada en la curiosidad disciplinada. Mientras que el desarrollador busca demostrar que el sistema funciona correctamente, el tester intenta descubrir situaciones en las que podría fallar.

Tomando como ejemplo un formulario de login, algunas preguntas que podría realizar un tester son:

- ¿Qué pasa si el usuario intenta iniciar sesión dejando vacíos los campos de usuario y contraseña?
- ¿Qué pasa si el usuario ingresa caracteres especiales o emojis en el campo de usuario?
- ¿Qué pasa si presiona el botón "Iniciar Sesión" varias veces seguidas muy rápidamente?

Estas situaciones permiten detectar errores que podrían no aparecer durante las pruebas habituales realizadas por el desarrollador.

Pichulman Miguel Angel

3. Contraste los tres enfoques fundamentales:

- o Pruebas de Caja Negra
- o Pruebas de Caja Blanca
- o Pruebas de Caja Gris
- o Para cada uno, defina su objetivo y dé un ejemplo de una prueba que solo se podría hacer con ese enfoque.

Enfoque	Objetivo	Ejemplo Exclusivo
Caja Negra	Verificar que el sistema haga lo que debe hacer desde la perspectiva del usuario.	Probar un cajero automático verificando únicamente las entradas y salidas, sin acceder a su código interno.
Caja Blanca	Verificar la lógica interna y la estructura del código.	Comprobar que todos los caminos de ejecución de una función Java hayan sido recorridos por pruebas unitarias.
Caja Gris	Combinar pruebas funcionales con conocimiento parcial de la arquitectura interna.	Registrar un usuario desde la interfaz y luego consultar directamente la base de datos para verificar si el registro se almacenó correctamente.

Pichulman Miguel Angel

Parte 2: Técnicas de Diseño de Pruebas de Caja Negra

Contexto: Analice las siguientes especificaciones de requisitos de un sistema:

- Requisito 1 (Campo "Edad"): "El sistema debe permitir registrar usuarios cuya edad esté entre 18 y 65 años, inclusive".
- Requisito 2 (Regla de Negocio): "El sistema aplicará un descuento del 15% solo si el cliente es 'Premium' Y la compra es 'mayor a \$50.000'".

Consignas:

1. Para el Requisito 1 (Campo "Edad"):

- Liste las 5 Particiones de Equivalencia (válidas e inválidas) que identificó.
- Liste los 6 casos de prueba que se derivan del Análisis de Valores Límite (BVA).

Particiones de Equivalencia:

- P1: Edad < 18 (Inválida)
- P2: $18 \leq \text{Edad} \leq 65$ (Válida)
- P3: Edad > 65 (Inválida)
- P4: Valor no numérico (Inválida)
- P5: Campo vacío (Inválida)

Análisis de Valores Límite:

Límite inferior (18)

Edad	Resultado esperado
17	Inválido
18	Válido
19	Válido

Límite Superior (65)

Edad	Resultado esperado
64	Válido
65	Válido
66	Inválido



Pichulman Miguel Angel

2. Para el Requisito 2 (Regla de Negocio):

- o Complete la Tabla de Decisión (mostrando las 4 reglas/combinaciones posibles).
- o Explique por qué probar solo el caso "Premium Y Compra > \$50.000" (la Regla 1) sería insuficiente.

Cliente Premium	Compra > \$50.000	¿Aplicar descuento?
Sí	Sí	Sí
Sí	No	No
No	Sí	No
No	No	No

Probar únicamente la Regla 1 permitiría verificar que el descuento se aplica correctamente cuando se cumplen ambas condiciones. Sin embargo, no comprobaría qué sucede cuando una o ambas condiciones no se cumplen.

Si solo se prueba ese caso, podrían existir errores como:

- Que el sistema aplique descuentos a clientes no Premium.
- Que aplique descuentos en compras menores a \$50.000.
- Que aplique descuentos siempre, independientemente de las condiciones.
- Que falle al evaluar alguna de las condiciones de la regla de negocio.

Por esta razón es necesario probar las cuatro combinaciones posibles de la tabla de decisión, garantizando que la lógica completa funcione correctamente y no solamente el caso esperado más favorable.

Pichulman Miguel Angel

Parte 3: Pruebas Estructurales y Niveles de Prueba

1. ¿Qué es la Cobertura de Código (Code Coverage)?

La Cobertura de Código es una métrica utilizada en las pruebas de Caja Blanca que indica qué porcentaje del código fuente ha sido ejecutado durante las pruebas. Su objetivo es medir qué tan bien están siendo probadas las diferentes partes del programa.

2. Dado el siguiente pseudocódigo:

```
Funcion es_mayor_de_edad(edad):
```

```
    IF edad >= 18:
```

```
        Imprimir("Es mayor")
```

```
    ELSE:
```

```
        Imprimir("Es menor")
```

o Explique por qué una prueba con edad = 25 (Prueba 1) es insuficiente, aunque el código "funcione". ¿Qué porcentaje de Cobertura de Decisión alcanza esa única prueba?

Una prueba utilizando el valor 25 únicamente verifica el camino donde la condición del IF es verdadera.

Esto demuestra que el sistema funciona correctamente para una persona mayor de edad, pero no permite saber qué sucede cuando la edad es menor a 18 años.

Por lo tanto, aunque el código parezca funcionar correctamente, todavía existe una parte de la lógica que no fue probada.

Solo se ejecuta la rama TRUE del IF.

- Rama TRUE: Probada
- Rama FALSE: No probada

Por lo tanto, la Cobertura de Decisión es del 50%.

o ¿Qué casos de prueba (qué dos valores) necesitaría para alcanzar el 100% de Cobertura de Decisión?

Para alcanzar el 100% deben ejecutarse ambas ramas del IF:

Caso de prueba	Resultado esperado
edad = 25	"Es mayor"
edad = 10	"Es menor"

Con estas dos pruebas:

- Rama TRUE → cubierta.
- Rama FALSE → cubierta.

Se obtiene una Cobertura de Decisión del 100%.

Pichulman Miguel Angel

3. Explique el propósito de la "Pirámide del Testing". Describa brevemente la diferencia entre Pruebas Unitarias, Pruebas de Integración y Pruebas de Sistema.

La Pirámide del Testing es un modelo que representa cómo deberían distribuirse los esfuerzos de prueba dentro de un proyecto de software. La idea principal es tener muchas pruebas pequeñas y rápidas en la base, y pocas pruebas complejas y lentas en la parte superior.

- **Pruebas Unitarias:** Son pruebas que verifican el funcionamiento de una única función, método o clase de manera aislada.
Ejemplo: comprobar que una función que calcula el IVA devuelve el resultado correcto.
- **Pruebas de Integración:** Verifican que distintos módulos o componentes trabajen correctamente juntos.
Ejemplo: comprobar que una API puede guardar y recuperar datos desde una base de datos.
- **Pruebas de Sistema:** Evalúan el sistema completo desde la perspectiva del usuario final.
Ejemplo: realizar todo el proceso de compra en una tienda online, desde la selección del producto hasta la confirmación del pago.

4. Diferencie entre Pruebas Funcionales y Pruebas No Funcionales. Dé un ejemplo de cada una.

- **Pruebas Funcionales:** Verifican que el sistema realice correctamente las funciones especificadas en los requisitos.
Ejemplo: comprobar que el botón "Iniciar Sesión" permita acceder al sistema cuando las credenciales son correctas.
- **Pruebas No Funcionales:** Evalúan características de calidad como rendimiento, seguridad, usabilidad o estabilidad.
Ejemplo: medir cuánto tiempo tarda el sistema en responder cuando existen 500 usuarios conectados simultáneamente.

Pichulman Miguel Angel

Parte 4: Aplicaciones en el Mundo Real y Preguntas para Investigar

1. Identifique una falla de software ("bug") famosa que haya tenido impacto en el mundo real. Describa brevemente qué pasó y especule: ¿Qué nivel de prueba (Unitario, Integración, Sistema) cree usted que falló o faltó? Justifique.

Un caso famoso fue la falla del cohete Ariane 5 Flight 501, ocurrida el 4 de junio de 1996. Aproximadamente 37 segundos después del despegue, el sistema perdió la información de guiado y control debido a errores en el software del sistema de referencia inercial, provocando la destrucción automática del vehículo. La investigación oficial concluyó que también existieron deficiencias en las pruebas realizadas sobre el sistema completo. Considero que las pruebas de integración y de sistema fueron insuficientes, ya que un escenario real de ejecución podría haber permitido detectar el defecto antes del lanzamiento.

2. Preguntas de Investigación (requiere información fuera del material proporcionado):

o Pregunta 1: Investigue y compare dos herramientas de automatización de pruebas de UI (por ejemplo, Selenium vs. Cypress). Explique brevemente cómo funciona cada una y cuándo sería apropiado usar una sobre la otra.

Selenium

Selenium es una herramienta de automatización de pruebas que permite controlar navegadores web mediante scripts. Es compatible con múltiples lenguajes de programación como Java, Python, C# y JavaScript. Se utiliza ampliamente para pruebas funcionales y de regresión en aplicaciones web.

Cypress

Cypress es una plataforma moderna para pruebas de aplicaciones web que ejecuta las pruebas directamente dentro del navegador. Está orientada principalmente al desarrollo frontend y facilita la creación y depuración de pruebas end-to-end.

Característica	Selenium	Cypress
Lenguajes soportados	Java, Python, C#, JS, entre otros	JavaScript y TypeScript
Navegadores	Gran variedad	Principalmente navegadores modernos
Facilidad de uso	Más complejo	Más simple e intuitivo
Velocidad de ejecución	Menor	Mayor
Ideal para	Proyectos grandes y multiplataforma	Aplicaciones web modernas

Selenium es recomendable cuando se necesita compatibilidad con varios navegadores, sistemas operativos y lenguajes de programación.

Cypress es recomendable cuando se desarrollan aplicaciones web modernas y se busca rapidez en la creación y ejecución de pruebas.

Pichulman Miguel Angel

o Pregunta 2: Investigue qué es el Testing de Regresión. Explique por qué es un candidato ideal para la automatización en las metodologías ágiles.

El Testing de Regresión es un tipo de prueba que se realiza después de modificaciones en el software para verificar que los cambios realizados no hayan afectado funcionalidades que ya funcionaban correctamente. Su objetivo es detectar errores introducidos por nuevas características, correcciones o refactorizaciones.

En metodologías ágiles se realizan cambios frecuentes y entregas continuas. Ejecutar manualmente todas las pruebas después de cada modificación sería costoso y lento.

La automatización permite:

- Ejecutar pruebas repetitivas de forma rápida.
- Detectar errores tempranamente.
- Reducir el tiempo de validación antes de cada entrega.
- Mantener la calidad del software durante ciclos de desarrollo cortos.

Por estas razones, el Testing de Regresión es uno de los principales candidatos para la automatización dentro de entornos ágiles y de integración continua.

Pichulman Miguel Angel

Fuentes:

Cypress.io. (2026). Cypress Documentation. <https://docs.cypress.io/>

IBM. (2025). What is regression testing? <https://www.ibm.com/think/topics/regression-testing>

Microsoft. (2024). Types of tests that implementation projects use: Regression testing.
<https://learn.microsoft.com/en-us/dynamics365/guidance/implementation-guide/testing-strategy-test-types>

Selenium Project. (s.f.). Selenium Documentation. <https://www.selenium.dev/>

European Space Agency. (1996, July 23). Ariane 501 – Presentation of Inquiry Board report. ESA.
https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report